Big-O Discussion

This project uses a mix of database operations, graph traversal, and a custom hash-based lookup layer. Most of the user-facing features feel fast because the expensive work is either done with indexed SQL queries or with data structures that reduce repeated searching.

Below are the main methods we discussed as a team and the time cost we expect from each one.

1. `MyUnoredredMap::contains(const Key& key)`

This method is used to check whether a key already exists in our custom unordered map before we continue with a stadium lookup.

- Average case: **O(1)**
- Worst case: **O(n)**

Why:

The key is hashed first, then the method only walks through the linked list in one bucket. If the hash values are spread out well, the bucket stays short and the lookup is close to constant time. If many keys land in the same bucket, the linked list in that bucket can grow and the check becomes linear.

Why it matters in our project:

This is the small "gatekeeper" step before going to the database for stadium lookups. The program first checks the custom map. If the key exists there, the lookup continues to the database using the stored mapping information instead of blindly searching from scratch.

2. `MyUnoredredMap::insert_or_assign(const Key& key, const Value& value)`

This method either updates an existing key or inserts a new one into the custom unordered map.

- Average case: **O(1)**
- Worst case: **O(n)**
- Worst case during a rehash: **O(n)**

Why:

The normal case is fast for the same reason as `contains()`: hashing sends the key to one bucket, and only that bucket is searched. When the load factor gets too high, the map grows and rehashes all existing entries into a new bucket array. That single resize step costs linear time, but it does not happen on every insertion.

Why it matters in our project:

This is what lets the custom map act as a middle layer between the GUI/repository side and the database side. Once the key is stored, future stadium checks can go through the map first before touching the database lookup.

3. `StadiumRepository::getAllStadiums(...)`

This method loads stadium rows from the database and returns them as a `vector<Stadium>`.

- SQL sorting cost: about **O(n log n)** in the general case
- Row materialization cost in C++: **O(n)**
- Combined practical view: **O(n log n)**

Why:

The result set may be sorted by team name, stadium name, league, date opened, seating capacity, distance to center field, typology, or roof type. The database does the heavy sorting work. After that, the program still has to walk through each returned row and build a `Stadium` object for it.

Why it matters in our project:

This method powers the main browse page. Whenever the user changes a filter or sorting mode, this is one of the main methods that refreshes the list.

4. `TripPlanner::computeShortestPaths(int start_id, ...)`

This is the core Dijkstra-style method used by several trip planning features.

- Time complexity: **O((V + E) log V)**
  In our graph, `V` is the number of stadiums and `E` is the number of distance connections.

Why:

The method uses a priority queue to always expand the current shortest-known stop first. Each useful push/pop from the priority queue costs log V, and the algorithm may examine many edges while relaxing distances.

Why it matters in our project:

This method is the backbone for:

- shortest trip from Dodger Stadium to a selected team
- custom ordered trips
- custom efficient trips
- visit-all trips that repeatedly choose the next closest stadium

Without this method, the trip planner would have to use much slower brute-force path checking.

5. `TripPlanner::buildShortestPath(...)`

This method rebuilds the final path after the shortest-path search is done.

- Time complexity: **O(k)**

Here, `k` is the number of stadiums in the final path from start to target.

Why:

It walks backward from the target using the `previous` map until it reaches the starting stadium, then reverses the list.

Why it matters in our project:

The shortest-path search finds distances, but this method is what turns those distances into the actual stadium-by-stadium route the user can see.

6. `ShoppingCart::addItem(const Souvenir& souvenir, int quantity)`

- Time complexity: **O(m)**

Here, `m` is the number of souvenir entries already in the shopping cart.

Why:

The cart stores items in a vector. Before adding a new entry, the method checks whether the same souvenir is already in the cart so it can increase the quantity instead of duplicating the item.

Why it matters in our project:

The cart is not using a hash map or tree. That keeps the code simple, but it means adding an item may require scanning the current cart contents.

7. `DatabaseManager::import_distances_csv_file(const QString& distances_csv)`

- Time complexity: about **O®** for reading and processing rows

Here, `r` is the number of rows in the CSV file.

Why:

The method reads the file row by row, normalizes stadium names, checks whether each stadium exists, parses the mileage, and then upserts the row into the database. Each row is handled once, so the overall time grows roughly with the size of the CSV.

Why it matters in our project:

This is one of the administrator-side methods that keeps the stadium graph current, especially when a new expansion team or new distance file is added.

Final Note

The most important performance idea in this project is that we are not relying on one single structure for everything.

- The custom unordered map gives us fast key checks before stadium lookups continue to the database layer.
- The database gives us persistence and filtered queries.
- The priority queue, queue, and stack make the graph algorithms practical.
- Vectors keep returned results simple and easy to display in the GUI.

That combination makes more sense for this project than forcing every feature into one structure.